

Introduction to Computer Programming 2024-25

Department of Business and Management
Giorgio Piccardo

Python working environment

Assembly

Assembly, or assembly language, is a low-level programming language that is closely related to a computer's machine code instructions.

Unlike high-level programming languages like Python or Java, which are more human-readable and abstract away the details of the hardware, assembly language instructions correspond directly to the machine instructions executed by the CPU.

Print “c” in assembly

```
section .data
    msg db 'c'      ; The character to print
    len equ 1       ; Length of the character

section .text
    global _start

_start:
    ; syscall: write(fd, buffer, length)
    ; write to stdout (fd = 1)
    mov eax, 4      ; syscall number for sys_write (Linux)
    mov ebx, 1      ; file descriptor (1 = stdout)
    mov ecx, msg     ; pointer to the message
    mov edx, len     ; length of the message
    int 0x80        ; interrupt to call kernel

    ; syscall: exit(status)
    mov eax, 1      ; syscall number for sys_exit (Linux)
    xor ebx, ebx    ; exit status 0
    int 0x80        ; interrupt to call kernel
```

Translator

Machine Language has a low level of abstraction, because it is closely related to the hardware that will execute the program.

For programmers, high-level languages are more convenient: they are more expressive, and closer to the way of thinking of a human being (e.g., C, C++, BASIC, Java, Python, javascript, . . .)

Print the character “c”

Assembly

```
section .data
    msg db 'c'      ; The character to print
    len equ 1       ; Length of the character

section .text
    global _start

_start:
    ; syscall: write(fd, buffer, length)
    ; write to stdout (fd = 1)
    mov eax, 4      ; syscall number for sys_write (Linux)
    mov ebx, 1      ; file descriptor (1 = stdout)
    mov ecx, msg     ; pointer to the message
    mov edx, len     ; length of the message
    int 0x80        ; interrupt to call kernel

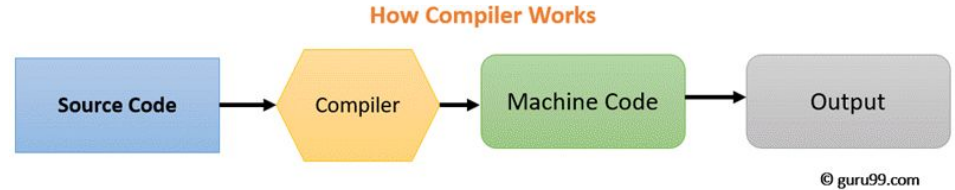
    ; syscall: exit(status)
    mov eax, 1      ; syscall number for sys_exit (Linux)
    xor ebx, ebx    ; exit status 0
    int 0x80        ; interrupt to call kernel
```

Python

```
print("c")
```

Compilers and interpreters

A compiler and an interpreter are both tools used to translate high-level programming languages into machine code, but they do so in different ways.



Compiler vs Interpreter

Compiler

A compiler translates the entire source code of a program into machine code before the program is executed.

This process results in an executable file, which can be run on the target machine without the need for the source code or the compiler.

Compilation happens once, and the resulting executable can be run multiple times.

Compilers typically perform various optimizations to improve the performance of the generated machine code.

Example languages that use compilers include C, C++, and Java.

Interpreter

An interpreter translates source code line-by-line or statement-by-statement and executes it directly.

This means the source code is translated and executed simultaneously, without producing a separate executable file.

Since interpretation happens at runtime, it can be slower than running compiled code because the interpreter needs to process each line every time the program is run.

Interpreters provide more flexibility and are often easier to use for scripting and prototyping because changes can be tested immediately.

Example languages that use interpreters include Python, Ruby, and JavaScript.

Compilers

Some languages are specifically designed to be compiled, such as: C, C++, Java, Pascal, C#, VB.NET, FORTRAN, COBOL. . .

Main advantages are:

- Execution speed (e.g., C may be ~100x faster than Python)
- It is possible to distribute only the executable (e.g., the .exe file in Windows):
 - It is not necessary to distribute the compiler
 - It is not necessary to distribute the source code, e.g. in order to protect intellectual property

Interpreters

Some languages are specifically designed to be interpreted, such as: classic BASIC and Visual Basic, Python, Javascript, Ruby, Perl, . . .

Main advantages:

- **Flexibility and ease of development:** since the interpreter translates commands one by one, when it translates a command it is aware of the execution context, i.e., of what happened since the program has been launched. Thus it is possible to design an interpreted language that accepts more concise, expressive instructions, whose actual meaning depends on the execution context. In a compiled language, the programmer usually has to specify more information through e.g. declarations.
- **Tighter development cycle:** programs can be executed immediately, without the additional step of compiling.

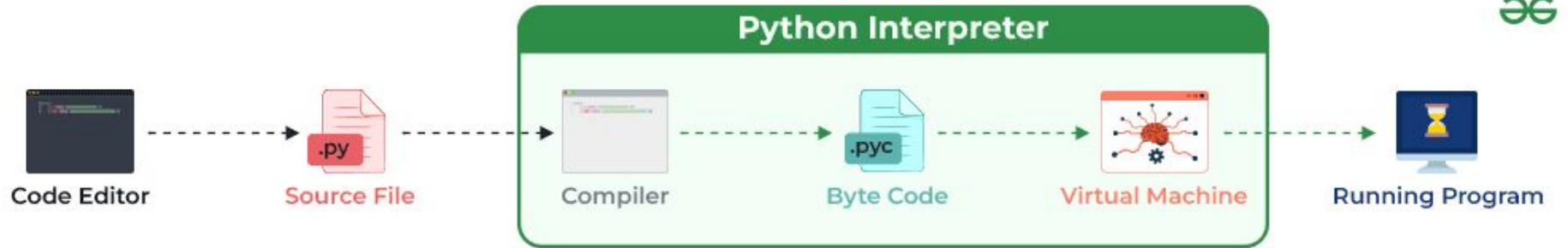
Interpreters

Main advantages (continued):

- **Interactivity:** it is possible to interactively send commands to the interpreter and immediately obtain the result, or interactively execute or modify portions of the program.

Thus, interpreted languages mainly focus on developer productivity, whereas compiled languages mainly focus on execution speed. It is possible to combine the best of both worlds, by developing performance-critical parts using a compiled language, and all the rest using an interpreted language.

How python works



How python works

Code Editor: This is the environment where you write your Python code. It could be any text editor or integrated development environment (IDE) like VS Code, PyCharm, etc.

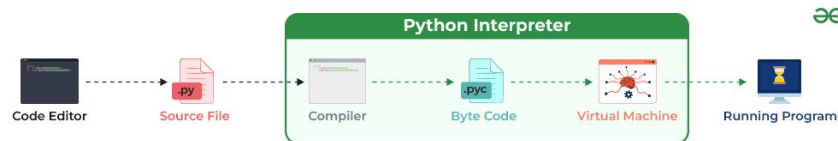
Source File (.py): After writing the code, it is saved in a file with a .py extension. This file contains the source code written in the Python programming language.

Compiler: When the Python interpreter runs the source file, it first compiles the source code into a lower-level form called bytecode. This bytecode is an intermediate representation that is platform-independent. The compiled bytecode is usually stored in .pyc files.

Byte Code (.pyc): This is the compiled version of the Python source code. The bytecode is not directly executable by the machine hardware but is an intermediate step that makes the code execution more efficient.

Virtual Machine (Python Interpreter): The bytecode is then executed by the Python Virtual Machine (PVM). The PVM is an interpreter that reads the bytecode and executes the corresponding machine-level instructions. This is where the actual computation and logic specified in the Python code take place.

Running Program: Finally, the result of executing the Python code is seen as a running program, which could be a command-line output, a graphical user interface, or any other kind of program output.



Download python

Visit the Python Website: Go to the official Python website: python.org.

Navigate to the Downloads Page: On the homepage, you will see a "Downloads" button. Hover over it to reveal a dropdown menu. It usually offers the latest version of Python for your operating system (e.g., Windows, macOS, Linux).

Select Your Operating System: Click on the appropriate link for your operating system. The website will automatically detect your OS and recommend the best version.

Download the Installer: Click the "Download Python [version number]" button to download the installer for your system.

Run the Installer: Once the download is complete, open the installer file.

- Windows: Run the .exe file. Make sure to check the box that says "Add Python to PATH" before clicking "Install Now."
- macOS: Open the downloaded .pkg file and follow the installation instructions.
- Linux: You can install Python using your distribution's package manager. For example, on Ubuntu, you can run `sudo apt-get install python3`.

Verify the Installation:

Open a terminal (Command Prompt on Windows, Terminal on macOS/Linux).

- Type `python --version` or `python3 --version` to check if Python was installed successfully. It should display the version number you installed.

Do not install python

Do not install the standard version of Python just yet. In the following slides, we will explore alternative methods that simplify the installation process, such as using Anaconda.

Interactive Mode

In interactive mode, you type and execute Python commands one at a time in an interactive shell. It's used for testing small pieces of code, experimenting, or learning Python.

How to Run: You open a Python shell (by typing `python` in the command line)

Output: The result of each command is immediately displayed after you press Enter.

Pros:

- Immediate feedback for each line of code.
- Great for experimenting, testing, and debugging small code snippets.

Cons:

- Not suitable for writing complex, long programs.
- Code typed in the interactive shell is not saved automatically, so it can't be reused easily.

Concept of Input and Terminal Output

Input: In the interactive mode, input is what you type at the prompt (`>>>`). For example, when you type `2 + 3` and press Enter, you're giving Python an instruction (input) to calculate the sum of 2 and 3.

Terminal Output: This is how Python responds to the input you provide. In the above example, after you press Enter, Python processes the input and displays the result 5 on the screen. This result is called the output.

Simple Example

Input: You type `print("Hello, World!")` and press Enter.

Output: Python shows Hello, World! in the terminal immediately.

Output environments

Python can produce output in a wide variety of environments beyond the terminal.

This includes integrated development environments (IDEs), Jupyter Notebooks for rich interactive content, graphical user interfaces (GUIs) for user-friendly applications, web browsers for dynamic web content, log files for background processes, and even direct hardware control for embedded systems. T

he output environment you choose depends on the purpose of your program and how you want users to interact with it.

Script mode

In script mode, you write Python code in a file (usually with a .py extension) and then run this file all at once. It's used for writing longer programs or scripts that need to be saved and reused.

How to Run: You run the script by executing the file through a command line (e.g., `python my_script.py`) or an IDE.

Output: The entire script runs from start to finish, and the output (if any) is displayed after the execution completes.

Pros:

- Ideal for writing complex programs.
- Code can be saved, modified, and reused.

Cons:

- Slower feedback since you need to run the entire script to see the result.
- Not ideal for quick testing or debugging small pieces of code.

Code editor

A code editor is a software tool that helps you write and edit code. For Python programming, a good code editor can enhance your coding experience by offering features like syntax highlighting, code completion, debugging tools, and more.

Basic Features of Code Editors

Syntax Highlighting: This colors different elements of your code (keywords, variables, strings) to make it easier to read and understand.

Code Completion: Offers suggestions and auto-completes code as you type, saving time and reducing errors (also AI Based).also i based

Error Highlighting: Highlights syntax errors or potential bugs in real time, helping you catch mistakes early.

Indentation Support: Automatically manages spaces and tabs for consistent code formatting, which is crucial in Python.

Popular Code Editors for Python

IDLE: Python's built-in editor that comes with the Python installation. It's simple and great for beginners, with features like a built-in Python shell, syntax highlighting, and a basic debugger.

VS Code (Visual Studio Code): A free, open-source editor from Microsoft that supports Python through extensions. It offers powerful features like IntelliSense (smart code completion), integrated Git, debugging tools, and a wide range of customizable extensions.

PyCharm: A feature-rich IDE specifically designed for Python development. It offers advanced features like code analysis, a powerful debugger, integrated version control, and support for web frameworks like Django and Flask.

Atom: An open-source editor by GitHub, which can be customized with a wide variety of packages. It offers features like syntax highlighting, Git integration, and cross-platform support.

Jupyter Notebook: An interactive coding environment often used for data science and machine learning. It allows you to write and run Python code in a web-based interface with support for rich media (images, charts).

What are IDEs

An IDE (or Integrated Development Environment) is a program dedicated to software development. As the name implies, IDEs integrate several tools specifically designed for software development. These tools usually include:

- An editor designed to handle code (with, for example, syntax highlighting and auto-completion)
- Build, execution, and debugging tools
- Some form of source control

Anaconda

[Anaconda](#) is a distribution of Python that comes with many additional tools and packages. Here's how it differs from a standard Python installation:

Bundled Tools and Libraries: Comes pre-packaged with over 1,500 packages, including popular data science and machine learning libraries.

Package Management: Uses conda, a powerful package manager that can install, update, and manage packages and their dependencies easily.

Ease of Installation: Provides an all-in-one installation that sets up Python, the conda package manager, and many useful packages.

GUI: Comes with Anaconda Navigator, a GUI that allows you to launch applications (e.g., Jupyter Notebook, Spyder) and manage environments and packages without using the command line.

Using Both Versions

After installation, you will have two versions of Python on your system: the one installed with Anaconda and the standard Python installation.

You can use the standard Python version via the command line (python or python3).

You can use Anaconda's version of Python by using the Anaconda Prompt or using Spider.

To avoid potential conflicts use only one of the two versions (Anaconda or Official Python).

Spyder

Spyder is an open-source Python IDE that's optimized for data science workflows. Spyder comes included with the Anaconda package manager distribution.

Anaconda is an open-source package and environment management system that runs on Windows, macOS, and Linux. It quickly installs, runs, and updates packages and their dependencies. It also easily creates, saves, loads, and switches between environments on your local computer

<https://www.anaconda.com/>

Online IDEs

Online Integrated Development Environments (IDEs) are web-based platforms that allow you to write, run, and debug code directly in a web browser without the need to install software on your local machine.

These IDEs provide a convenient way to code on the go, collaborate with others, and work in different programming languages, including Python.

Features of Online IDEs

Code Editing: They offer a code editor with features like syntax highlighting, code completion, and error checking.

Compilation and Execution: You can write and execute code directly in the browser. This includes support for multiple programming languages.

Debugging Tools: Some online IDEs come with built-in debugging tools to step through code, inspect variables, and set breakpoints.

File Management: You can create, edit, and manage project files within the IDE.

Collaboration: Many online IDEs support real-time collaboration, allowing multiple users to work on the same codebase simultaneously.

Environment Configuration: Some online IDEs offer pre-configured environments for different languages and frameworks, reducing the need to set up environments manually.

Popular Online IDEs for Python

Replit:

- Supports multiple languages, including Python.
- Offers features like real-time collaboration, a built-in terminal.
- Great for quick experiments, learning, and small to medium-sized projects.

Google Colab:

- Particularly popular for Python, especially in data science and machine learning.
- Based on Jupyter Notebooks, it provides an interactive environment for writing and running Python code.
- Free access to cloud-based resources, including GPUs.

GitHub Codespaces:

- Offers a full VS Code environment in the cloud, tightly integrated with GitHub repositories.
- Suitable for large-scale projects and professional development.

PythonAnywhere:

- Tailored for Python developers.
- Offers a cloud-based Python environment with support for web development frameworks like Django and Flask.
- Includes scheduled tasks, database support, and the ability to host websites.

JupyterHub:

- An online, collaborative version of Jupyter Notebooks.
- Widely used in educational settings for teaching Python, data science, and scientific computing.

IDEs for this course

You can use the IDE of your choice, but it is suggested not to use those with AI assistance because, at this stage of learning, it won't help you, and you won't be able to use it during the exam.

Using AI to improve your Python

You can ask the AI to provide analogies or examples to clarify a complex idea, which can help you understand it better. After that, try to implement the examples on your own to reinforce your learning.

Ask the AI for custom quizzes, reading lists, or practice exercises based on the topics you're finding challenging. For example, you might request "practice problems for Python loops" to get more targeted practice.

Use AI to explain common Python errors (e.g., `IndentationError`, `TypeError`). Learn why these errors occur and how to avoid them in the future, improving your coding skills.

Have the AI review your code for readability, efficiency, and best practices. Use the feedback to refine your code and develop better coding habits.